CSI247:
Data Structures

Lecture 15 – Queue ADT implementation

Department of Computer Science
B. Gopolang (247-275)

Previous lesson

a) Linked List

- A list of elements joined via links
- Each element represented as a Node with:
  - Data part
  - Link part (A pointer to next node in the list)
- Space for elements is dynamically managed
  - Additions: more space added
  - Removals: garbage collector re-claims used space

UNIVERSITY of BOTSWANA

b) LinkedList implementation

- Important to keep track of these:
  - Head - first element in the list
- 2 classes : Node class & LinkedList class
- Various operations, e.g.:
  - Check size & if empty
  - Add elements: in front, middle or end
  - Remove elements: from front, middle or end
  - Search for a specific element

Today …

Implementing our own Queue class

Lecture Objectives

- By the end of this lecture you must be able to:
  - Queue Implementation approaches:
    - Using an array
    - Using a LinkedList

1. Queue Review

- A linear data structure
  - Uses FIFO: First In First Out!
  - Has front/head & tail/rear
- Inserting an element
  - AKA **enqueue**
    - Elements join at the tail
- Removing an element
  - AKA **dequeue**
    - Elements exit from front
- Many queue application areas, e.g.
  - printer queue, keystroke queue, CPU processes, etc.

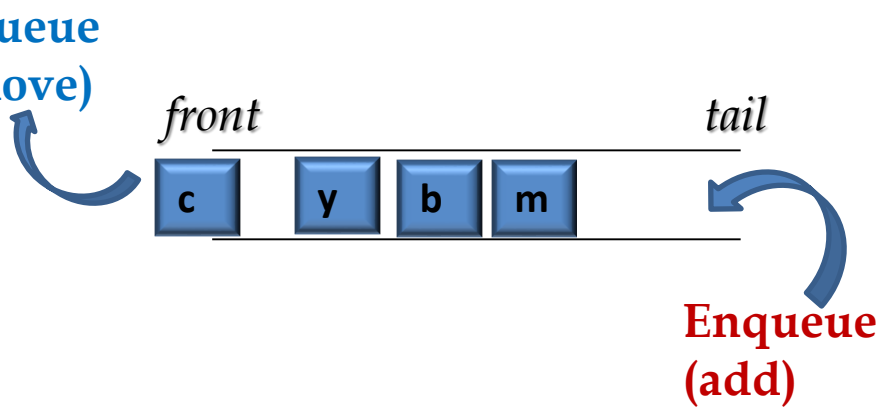
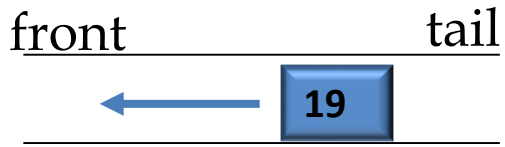


Illustration of Queue methods

Assuming an empty queue:

a) enqueue (19)



b) enqueue (28)

c) enqueue (35)

d) dequeue()

Exercise 1

a) Perform the following operations on an empty Queue:

enqueue(52), enqueue(96), enqueuer(63), dequeue(), enqueue(28), enqueuer(86), dequeue(), enqueue(69), dequeue()

b) What is the sequence of dequeued values?

c) What would be the queue length after performing the above sequence of operations?

d) Which item will be in front? At the tail?

Types of queues

a) Normal queue (FIFO) – standard queue

b) Circular Queue

- Last element is connected to the first element, forming a circle

c) Double-ended Queue (Deque)

- A double-ended queue
- Insertions and deletions allowed at both front & tail

d) Priority Queue

- More specialized type
  - Similar to Queue
    - Has front and tail
    - Removals from the front
  - BUT
    - Items are ordered by key value
      - Item with the lowest/highest key is always in front
      - Insertions done to maintain ordering
- Used in multitasking operating system.
- Normally represented using **heap** data structure, but can also use array & linked list

2. Java's Queue Implementation

- Rem:
  - An interface, in **java.util**
  - Declaration**:**

    public interface Queue<E> extends Collection<E>
  - **MUST** be implemented by a concrete class
- **Examples:**
  - PriorityQueue:
    - Queue<Integer> pq = new PriorityQueue<Integer> ();
  - LinkedList:
    - Queue<String> list = new LinkedList<String>();
  - ArrayDeque:
    - Queue<Student> ad = new ArrayDeque<>();

Queue methods in Java Implementation

- Rem:
  - Queue inherit from Collection
    - Hence, it includes some of methods from Collection
    - And adds its own method(s) (peek())
- Commonly used Queue methods:
  - **add()** – Insert element into a queue.
  - **remove()** – Removes and returns element at the front
    - Throws an exception when queue is empty
  - **peek()** - Returns element at the front of the queue
    - Not defined in Collection interface!

User-Defined Queue Implementation

- 2 options

a) Using an array
   - Working with fixed-size structure (disadvantage)
   - Remove from front, insert at the back (tail)
   - Careful: Overflow!!! (ArrayIndexOutOfBounds)

b) Using a Linked List
   - We use a Singly LinkedList
   - No size restriction
     - Queue will grow & shrink dynamically

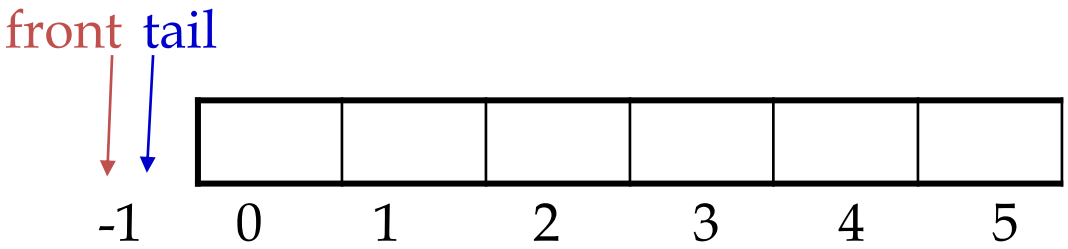
UNIVERSITY of BOTSWANA

- **Common Queue operations to implement**
  - **enqueue()**: **adds** an element (Rem: at the tail!)
  - **dequeue()**: **removes** & return front element
  - **peek()**: returns front element
    - Item is NOT removed!!!
  - **size()**: returns no. of elements in the queue
  - **isEmpty()**: Checks if queue is empty
    - Cannot dequeue when empty!
  - **isFull()**: Checks if queue is full
    - Cannot enqueue when full!

a) Array-based Implementation

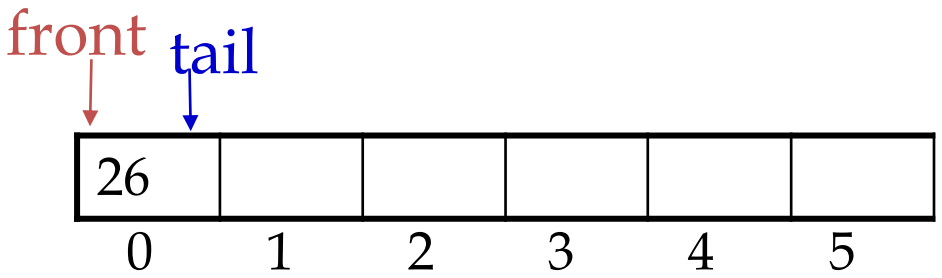
Note:

- MUST keep track of 2 positions
  - front
  - tail
- Initially, queue is empty, they MUST point to same index
  - Must be < 0 (smaller than smallest array index)
    - We will use -1

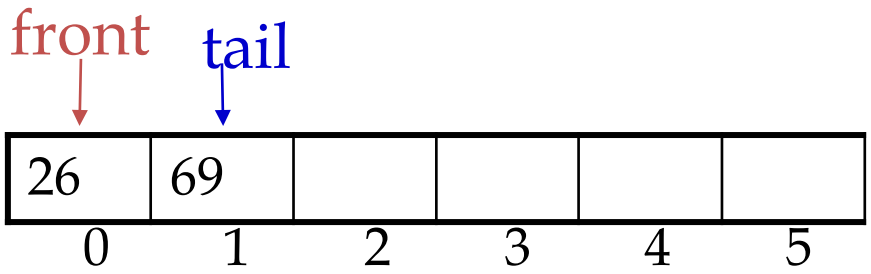


Effect of enqueue() on front and tail?

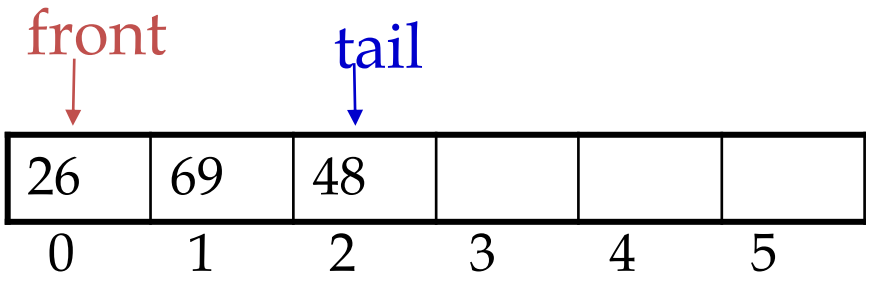
enqueue(26): 1st element:



enqueue(69): 2nd element:

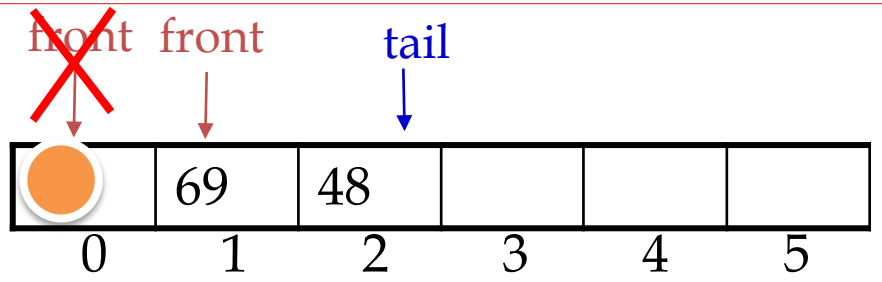


enqueue(48): 3rd element:

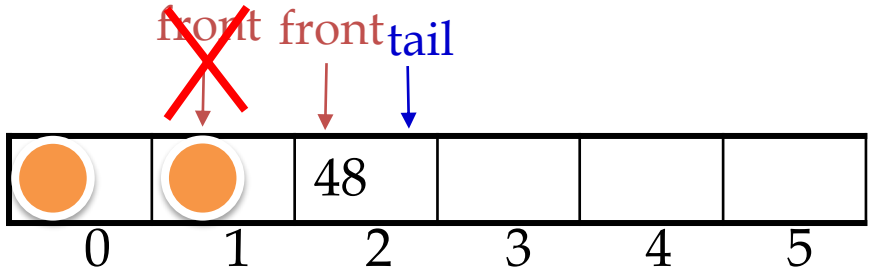


Effect of dequeue() on front & tail?

1st dequeue():



2nd dequeue():



- **Note:**
  - dequeue() removes item from the queue, **NOT the array**
    - Happens in **front**
    - **front** will keep on increasing until it reaches **tail**
    - Simultaneously, size will keep on reducing until 0 (**Empty!**)
      - At this point: both front & tail are set to 0 at next enqueue()

Our initial implementation of the Queue

- Let us call our class **Queue1**
- It will store String values for simplicity

```
public class Queue1 {
        // instance variable
        private String[] queue;  // An array to store queue elements
        private int front;       // for indexing front element
        private int tail;        // for indexing tail element
        // Rest of methods - to be added as we build the class
}
```

Writing Queue methods

i. Constructor

- Creates array of a specified size
- Set queue spaces to array size
- Initialises **front** & **tail** to -1

```
// for empty queue
    public Queue1(int n){
        queue = new String[n];
        front = -1;
        tail = -1;    }
```

ii. size()

- Returns number of elements in a Queue
  - Take-home: Loop from front to tail node, count nodes individually!

```
public int size(){
    // ...your code here!
}
```

iii. isEmpty()

- Checks if Queue has elements or not
- Method MUST be invoked before removing and peeking

```
public boolean isEmpty(){
    return size() == 0;
}
```

iv. isFull()

- Checks if Queue is full
  - A must: (Rem: fixed array size?)
- MUST be invoked before adding elements

```
public boolean isFull(){
    return size() == queue.length;
//all array slots occupied
}
```

v. enqueue()

- Adds an element to the queue
- Rem:
  - Additions: **at the rear**
  - We cannot enqueue when Queue is **full! (Arrays!)**

```
public void enqueue(String elem) {
  if (isFull())
      System.out.println("Full!!");
  else {
    if (isEmpty())
      front = tail = 0; // index=0
    else
      tail++;  // advance tail
    queue[tail] = elem; //store data
  }
}
```

vi. dequeue()

- Removes & returns element from front
  - Rem: Queue MUST have some elements!

```
public String dequeue()  {
   if (isEmpty()) {
      System.out.println("Queue
        empty!");
      return null;
   }
   else {
      String ele = queue[front];
      front++;
      return ele;
   }
}
```

vii. peek()

- Returns element in front, or null if there is none

```
public String peek(){
    if (!isEmpty())
        return queue[front];
    else
        return null;  // Queue empty (no front element)
}
```

viii. contains()

- Accepts a String value, and returns true when found, and false otherwise

```
public boolean contains (String key) {
    if (isEmpty())
        System.out.println("Empty.");
    else {
        int i=0;
        while (i<n) {
            if (queue[i].equals(key))
                return true;
            i++;
        } //while
    } //else
    return false;
}
```

Incomplete sample tester class

```
Queue1 q= new Queue1(4);
 // add elements
q.enqueue("CSI247");
e.enqueue("CSI243");
q.enqueue("CSI262");
q.enqueue("CSI142");
System.out.println(q1.size());
System.out.println(q1.isEmpty());
q1.enqueue("CSI141");
System.out.println(q1.size());
System.out.println(q1.peek());
```

```
q.dequeue(); // remove elements
q.dequeue();
q.dequeue();
System.out.println("Size: "+ q.size());
// search
System.out.println("Found: "+
    q.contains ("CSI247"));
```

Exercise 2

a) Type all code segments provided into a Java file

b) Compile the program.

- Fix errors, if any

c) Implement a tester class and test all methods implemented in Queue class

d) Implement an array-based Queue that works with data of any valid data type

- Generics!!

b) LinkedList-based implementation

- Rem:
  - LinkedList is a collection of Nodes
    - Always create a Node before inserting new element!
  - With LinkedList, there is no:
    - Size restriction: List grows and shrinks dynamically
    - Queue capacity
    - isFull()
  - As a queue:
    - Insert: At the tail
    - Remove: At the front

- Let us call our class **Queue2**
- We will store String values for simplicity

```
public class Queue2{
    private class Node {
        // ... Node instance variables
        // ... Node methods
    }
    // ... Queue2 instance variables
    // ... Queue2 methods
}
```

- **Node** class
  - Rem: LinkedList stores Nodes
  - We need this inner class!

```
private class Node{
    String data;   // node's data part
    Node next;     // a pointer to next node in the Queue
    Node (String data) {
      this.data = data;
      this.next = null;  // the last Node in the list
    }
}
```

- **Queue** class
  - Common Queue operations (methods) to implement:
    - **enqueue()**: inserts an element (back)
    - **dequeue()**: removes an element (front)
    - **peek()**: returns the element in front of queue
      - Item is NOT removed!!!
    - **isEmpty()**: Checks if queue has elements
      - Cannot dequeue if empty!
    - ~~**isFull()**: Checks if queue is full~~
    - **size()**: returns length of the queue
    - **contains()**: checks existence of some item in the queue

Implementing Queue members (instance variable and methods)

i. **Instance variables** (For storing data)

```
public Node front;   // tracks front node
public Node tail;    // tracks last node
```

ii. **Constructor** (For creating an empty Queue2 object)

```
public Queue2(){
    front = null;       // Initially, queue is empty!
    tail = null;
}
```

Initial look at our implementation

```
public class Queue2{
    private class Node{
        String data;       // node's data part
        Node next;   // a pointer to next node in the Stack
        Node(String data) {
          this.data = data;
          this.next = null;  // assumption: it will be the last Node
        } }
      // instance variables
      public Node front;   //  keeps track of front node
      public Node tail;     // keeps track of last node
      public Queue2(){ … }
     // Rest of the methods: to be added incrementally as we go along
}
```

Writing the rest of Queue methods

iii. isEmpty()

- Checks if Queue has elements or not

```
public boolean isEmpty(){
    return front == null;
}
```

iv. size()

- Returns number of nodes in the Queue

```
public int size(){
    // count nodes here!
}
```

v. enqueue()

- Adds an element to the Queue

```
public void enqueue(String data) {
    Node oldTail = tail;
    tail = new Node(data);   // new tail
    if (isEmpty())
        front = tail;     // 1st Node
    else
        oldTail.next = tail;
    }
```

vi. dequeue()

- Removes element from queue & and returns it

```
public String dequeue() {
    String data = front.data;
    front = front.next;
    if (isEmpty())
        tail = null;
    return data;
}
```

vii. peek()

- Returns front element in the Queue
  - Does not delete it!

```
public String peek() {
    if (!isEmpty())              // can not read from an empty Queue
        return front.data;
    else
        return null;             // No element in the queue
  }
```

viii. printQueue()

- For printing all elements in the queue (Rem: FIFO!)

```
public void printQueue(){
    if (!isEmpty()){
        Node currentNode = front;        // start from front
        while (currentNode != null){
            System.out.print(currentNode.data + " ");
            currentNode = currentNode.next;   //  advance to next
        }
    }}
```

Incomplete sample tester class

```
Queue2 que = new Queue2();
  // add elements
   que.enqueue("CSI247");
   que.enqueue("CSI243");
   que.enqueue("CSI261");
   que.enqueue("CSI141");
   que.enqueue("CSI142");
System.out.println("Front has: " + que.peek());
  que.enqueue("CSI392");
   que.dequeue(); // remove elements
   que.dequeue();
   que.dequeue();
   System.out.println("Queue size: "+ que.size());
// Implement these new methods
```

- contains(String) // returns true when a given element is found in the queue, otherwise returns false
- que.printFront(); // print 1st item
- que.printTail(); // print last item

Exercises

a) Type all code segments provided into a Java file

b) Compile it.

- Fix errors, if any

c) Implement a tester class and test all the methods

d) Implement a generic LinkedList-based Queue class

e) Implement a tester class to test methods implemented in d) above, using UB student names

f) Implement another tester class to test methods implemented in d) above, using Dog class objects

Summary

- Queue:
  - Data stored uses FIFO technique
    - Entry: occurs at the end
    - Removals: in front
- Queue implementation options
  - Using an array
    - Need to worry about capacity
  - Using a LinkedList
    - Permit dynamic additions and removals without worrying about capacity

Q & A